



BoilerSpace

CS 30700: Design Document

Team 3:

Aagam Bapna

Sheehan Dandapat

Rohan Potta

Nathan Wan

Jed Macapinlac

Bidit Acharyya

Index

● Purpose	2
● Design Outline	3
○ High Level Overview	
○ Interactions between System Components	
○ Diagram	
● Design Issues	10
○ Functional Issues	
○ Non-functional Issues	
● Design Details	15
○ Class Level Design	
○ Descriptions of Classes and Models	
○ Backend Controllers	
○ Frontend Controllers	
○ Sequence Diagrams	
○ UI Mockups	
○ Database Design	
○ API Routes	
○ Navigation Flow Map	

Purpose

Studying for exams can be hard, especially for someone who has just entered college or someone new to a certain class. It is a daunting task not only to find friends, but also to learn the content of an upcoming exam without any peer advice or guidance.

Therefore, the idea of creating a single source of truth for students of any major or class emerged as a way to reduce isolation and improve academic collaboration.

BoilerSpace is exactly that: a web-based application designed to eliminate the fear of being alone during exam season. By connecting students to available study spaces, nearby peers, and shared course resources in an accessible and reliable way, it breaks the information barrier students face when searching for class content while also providing an inviting platform for collaboration toward a common goal. Equipped with an AI that parses through shared notes and documents for a specific class, our website provides pooled insights from thousands of students over the history of that course and can also generate high-quality, practice-based problems.

By reducing the time spent searching for study space, improving the discoverability of peer-run sessions, and preserving collective knowledge across meetings, BoilerSpace aims to reduce friction in academic collaboration, increase student engagement, and strengthen the on-campus learning community.

Design Outline

High Level Overview:

BoilerSpace is a web-based application that enables Purdue students to access core features such as real-time crowdsourced room availability data, course content submitted by students throughout the history of a course, AI-powered note querying and summarization assistance, and the ability to schedule meeting times with fellow classmates. This will be implemented using the client-server model, in which one server simultaneously handles access from a large number of clients by operating concurrently. The system will be built using the MERN (MongoDB, Express, React, Node.js) stack: a React-based client interface, a Node.js/Express backend server, a MongoDB database for persistent storage, and integration with an external AI service for note querying and summarization.

Purpose of each Component:

1. Client
 - a. Provides users with an interactive interface to view room availability, study sessions, and upload or search study materials.
 - b. Sends asynchronous HTTP (REST) requests to the server.
 - c. Receives and interprets JSON responses from server and renders into the user interface.
2. Server
 - a. Receives and handles HTTP requests from clients.
 - b. Validates requests and sends queries to the database to add, modify, or retrieve data as needed.
 - c. Aggregates real time room updates and manages study session coordination.
 - d. Communicates with external AI API to process user queries related to uploaded study materials.
 - e. Make JSON responses and send them back to the clients.

3. Database

- a. MongoDB database stores all data used in applications such as user information, building and room data, study session records, and uploaded notes.
- b. Responds to queries from the server and returns requested data for processing.

Interactions between System Component:

Client to Server Interactions:

The client sends HTTP requests to the server whenever a user performs any action. This includes logging in, registering a new account, viewing the map, searching for buildings, checking into rooms, creating study sessions, joining sessions, uploading notes, sending messages, adding friends, RSVPing to events, updating their profile, changing settings, creating clubs, and browsing courses. Every button click or form submission triggers a request from the client to the server.

Server to Client Interactions:

The server responds to every client request with JSON formatted data. When users log in, the server sends back authentication tokens and user profile data. When viewing the map, the server sends building locations and room availability. When browsing sessions, the server sends session details, participant lists, and times. When checking notifications, the server sends all unread alerts. The server also sends error messages when requests fail or validation errors when users submit invalid data.

Server to Database Interactions:

The server queries the database to create new records when users register accounts, check into rooms, create study sessions, upload notes, send messages, or schedule events. The server queries the database to read existing data when users view their profile, browse available rooms, search for courses, look at session participants, read messages, or check their friends list. The server updates database records when users edit their profile, change session details, update room status, modify event information, or adjust notification settings. The server deletes from the database when users remove friends, leave sessions, delete messages, cancel events, or delete their account.

Database to Server Interactions:

The database returns user records when the server authenticates login attempts. The database sends building and room data when the server requests location information. The database provides course lists when users browse their enrolled classes. The database returns session information including participants, notes, and discussion threads. The database sends friendship connections when displaying friend lists. The database provides club and event data for organization pages. The database confirms successful creation, update, or deletion operations. The database returns error messages when queries fail or constraints are violated.

Server to AI Service Interactions:

The server sends user queries to the AI service when students ask questions about their notes. The server uploads course materials and notes to the AI service for indexing and embedding creation. The server requests study guide generation by providing course content and learning objectives. The server asks for practice question generation based on uploaded materials. The server submits summarization requests for long documents or session notes. The server provides context from multiple note sources when users ask complex questions.

AI Service to Server Interactions:

The AI service returns answers to user questions with citations to source materials. The AI service sends back generated study guides with structured content and key topics. The AI service provides practice questions with varying difficulty levels. The AI service returns document summaries and key point extractions. The AI service sends vector embeddings for uploaded materials that get stored in the database. The AI service indicates when it cannot answer questions due to insufficient source material.

Server to Server Internal Interactions:

The authentication module validates user credentials and generates session tokens. The notification service triggers alerts when sessions start, friends join sessions, or events approach. The validation layer checks all incoming data before database operations. The file upload handler processes and stores note documents and images. The availability calculator aggregates check-in data to compute room capacity and crowd levels. The search indexer updates searchable content when new notes or sessions are created.

Client Internal Interactions:

The React frontend updates the user interface when server responses arrive. The map component refreshes room availability indicators based on real-time data. The notification bell updates its count when new alerts arrive. The session list re-renders when users join or leave sessions. Form validation runs before sending requests to the server. The navigation system updates the current page based on user clicks.

Database Internal Interactions:

MongoDB maintains relationships between users and their friends. The database links sessions to courses, rooms, and participants. The database connects notes to sessions and courses for organization. The database associates messages with threads and discussion participants. The database tracks which users belong to which clubs and have permissions. The database enforces constraints like preventing duplicate friendships or invalid check-ins.

Cross-Component Notification Flow:

When a user creates a session, the client sends the request to the server, the server stores it in the database, the server triggers the notification service, the notification service queries the database for relevant users, the server sends notifications to those users, and their clients display the alerts.

Cross-Component Authentication Flow:

When a user logs in, the client sends credentials to the server, the server queries the database for the user record, the database returns the hashed password, the server validates the password, the server generates an authentication token, the server stores the session in the database, and the server sends the token back to the client for future requests.

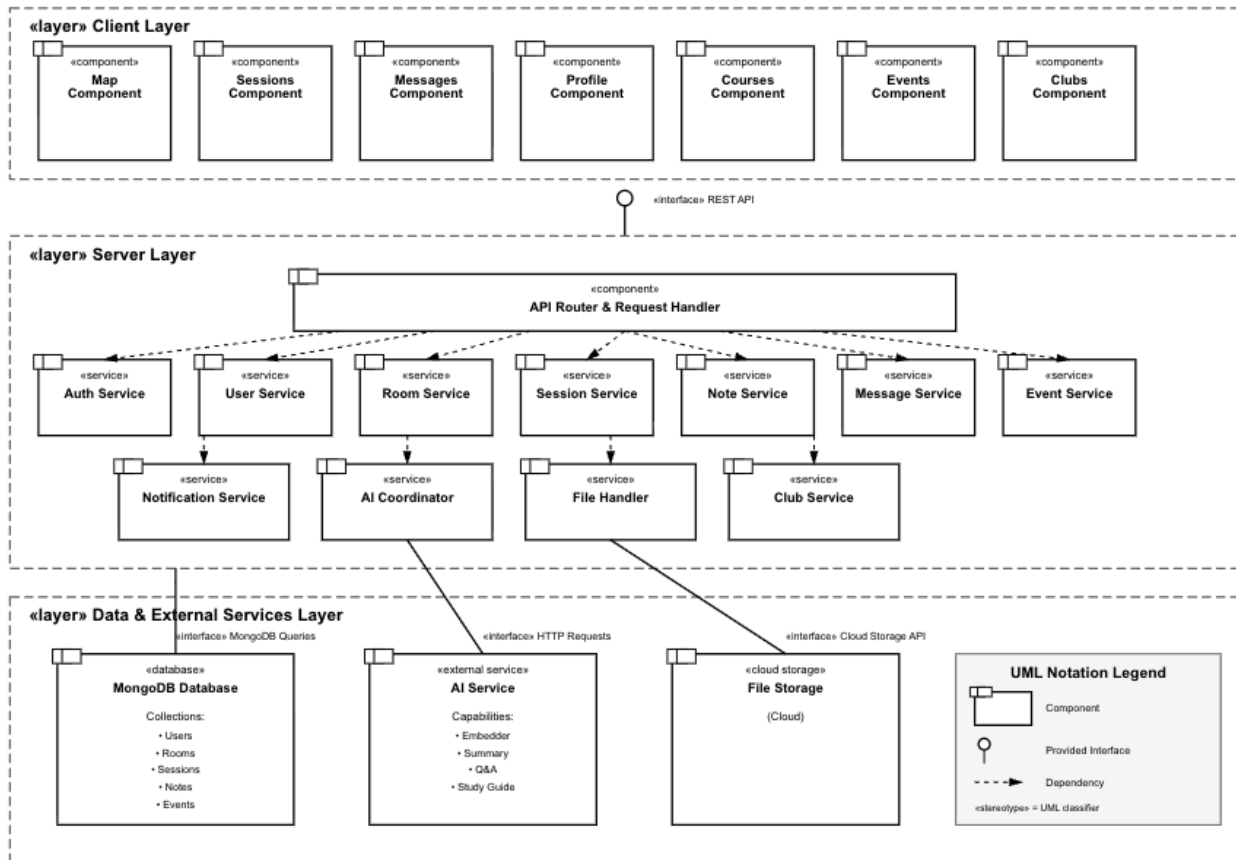
Cross-Component Real-Time Update Flow:

When a user checks into a room, the client sends the check-in to the server, the server creates a CheckIn record in the database, the server calculates updated availability metrics, the server broadcasts the update to other connected clients viewing that building, and those clients refresh their room availability displays.

Cross-Component Content Upload Flow:

When a user uploads notes, the client sends the file to the server, the server validates the file type and size, the server stores the file in cloud storage, the server creates a Note record in the database, the server sends the note content to the AI service for indexing, the AI service returns embeddings, the server stores the embeddings in the database, and the server confirms the upload to the client.

Diagram



Design Issues

Functional Issues:

1. How should we determine what rooms are available to study in?
 - Option 1: Use the capacity of the room only
 - Option 2: Utilize self-reported data from users
 - Option 3: Combine self-reported data with scheduled patterns (e.g. weekly club meetings in a room)

Choice: Option 3

Justification: Using the capacity of the room is not the best way to determine room availability as the number is static and doesn't account for other circumstances, such as a large portion of the room will be taken up by extra props. The benefit of using self-reported data is that users can account for this when reporting capacity. Combining this with scheduled patterns, such as knowing that a room will always be occupied at 6:00PM on a Monday for a club meeting, is the best way to determine room availability.

2. Should study sessions be public or private?
 - Option 1: All sessions should be public
 - Option 2: All sessions should be public
 - Option 3: The visibility of the session is up to the user

Choice: Option 3

Justification: Having a "one size fits all" solution is not ideal for users as each user will have differing needs from our app. Allowing users to choose the visibility of their study sessions gives them flexibility, as some students prefer open collaboration while other students prefer private study sessions with people they know.

3. How should user friendships be framed?

- Option 1: Use social media following system
- Option 2: Mutual friends get full visibility of each other's profiles

Choice: Option 2

Justification: The social media following system does not align with the goals of our app because the main purpose is to encourage and facilitate collaboration among students, and the social media following system would undermine this by turning the app into a social networking platform. Giving full visibility to mutual friends encourages continued collaboration between users and also keeps privacy.

4. How should study material uploads be moderated?

- Option 1: User voting system (like Reddit) with the ability to report inappropriate posts
- Option 2: No moderation for posting
- Option 3: All posts must be checked by admin before becoming visible

Choice: Option 1

Justification: There needs to be moderation for posting to ensure that all posts meet a minimum quality threshold. No moderation leads to potentially inappropriate posts that go against the purpose of our app. Having admin manually check every post is time-consuming and inefficient, so using a democratic process like a user voting system simplifies this process and also makes the app more user-friendly.

5. How should we implement AI for study material querying?

- Option 1: Allow the AI to answer using general knowledge from the internet
- Option 2: Implement a RAG pipeline so it uses the uploaded course material to generate responses
- Option 3: Have a filter so that the AI can only answer questions if they contain exact matches to a predetermined list of words

Choice: Option 2

Justification: Unrestricted AI responses using general knowledge from the internet may include hallucinations and incorrect answers, or solutions that do not match the content taught in a user's course, so it isn't an ideal solution.

Additionally, requiring the user's query to contain matches to a bank of predetermined phrases is not efficient or user-friendly. Utilizing a RAG pipeline is the best solution as the answers will be based off of course material fed by the user, which minimizes incorrect answers and ensures answers match the course content.

Non-Functional Issues:

1. How can we scale the app?

- Option 1: Use a single monolithic server
- Option 2: Utilize horizontally scalable cloud deployment

Choice: Option 2

Justification: In order to handle events of high user traffic, such as around finals or midterms when many students will be looking for study spots at the same time, we need to choose an architecture to best handle this. Using a horizontally scalable cloud deployment will help our app handle high traffic without sacrificing performance.

2. How should we store sensitive user data?

- Option 1: Store in the database as is
- Option 2: Hash the data using encryption methods

Choice: Option 2

Justification: Storing sensitive information as it is not safe as it keeps user data exposed in the event of an attack. Hashing the data using encryption methods adds a level of security so that users are protected.

3. How should real-time communication be implemented?

- Option 1: HTTP Polling
- Option 2: Node.js based event handling via Socket.IO
- Option 3: Use a third-party real-time service like Firebase Realtime Database

Choice: Option 2

Justification: HTTP polling is inefficient as repeated HTTP requests harm performance especially when requests are being made when no messages are being sent. Relying on a third-party service reduces control on our ends and some also cost money for full usage. Option 2 is the most optimal because Socket.IO uses WebSockets for communication, and uses HTTP polling as a failsafe.

4. Which frontend framework should we use?

- HTML + CSS + JavaScript
- React
- Angular
- Vue.js

Choice: Option 2

Justification: React is the best option out of all the above because it provides component-based architecture which gives us flexibility as developers and will help us to create features like a user dashboard, an interface for the AI study helper, and more. Additionally, it is easy to integrate with Node.js, which we are using for scalability purposes, making it well-suited for our app.

5. How should uploaded files be stored?

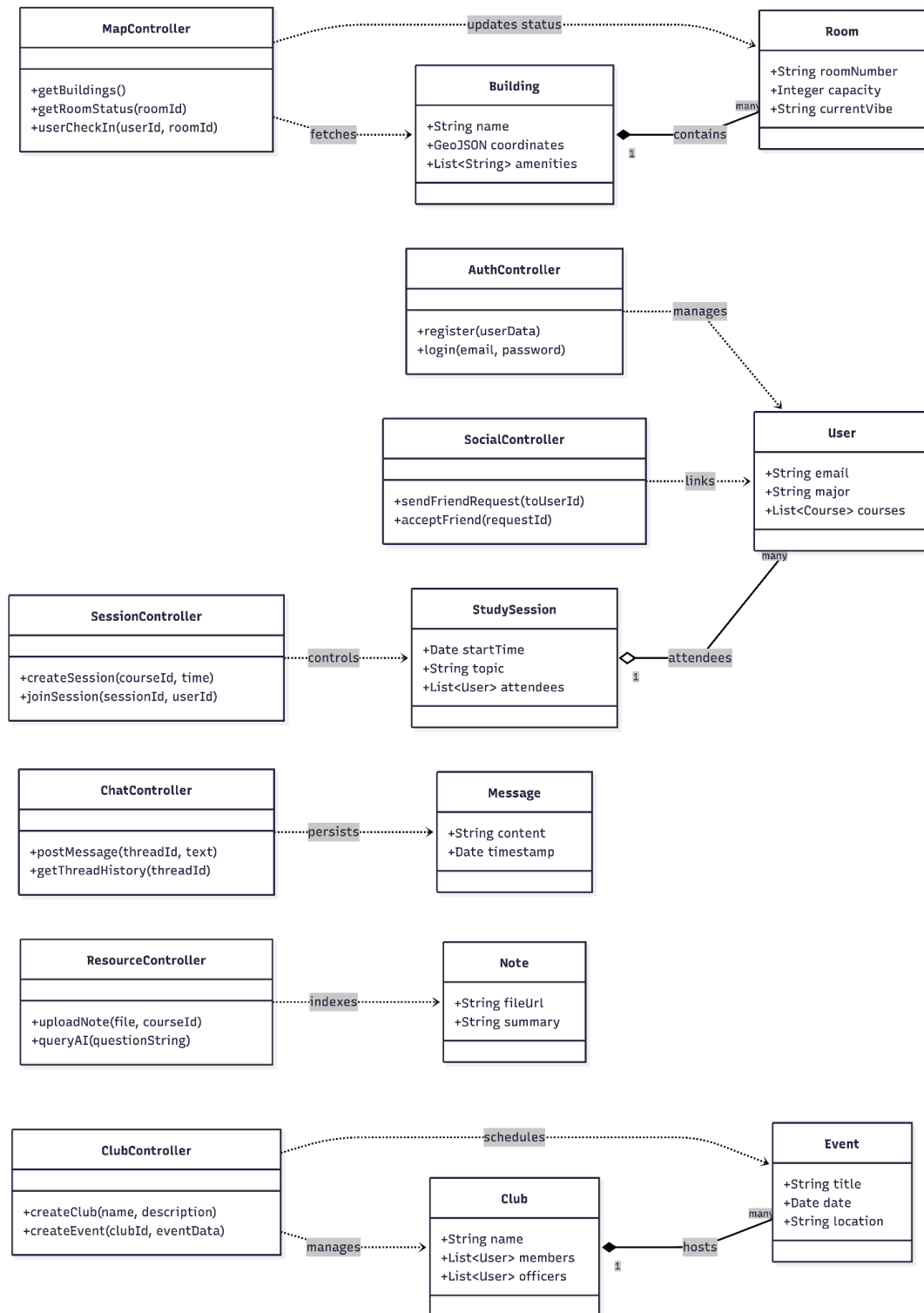
- Option 1: Use a cloud object storage system like Google Cloud Storage and store metadata in MongoDB
- Option 2: Store directly in MongoDB using GridFS
- Option 3: Store locally on the application server

Choice: Option 1

Justification: Storing files directly into MongoDB is not a viable option as large file uploads can lower performance as files larger than 16 MB would need to be split using GridFS, increasing the complexity of the database as well. Storing locally makes it hard to scale the app. Using Google Cloud Storage as it makes it easy to scale our app and also provides up to 5 GB of free storage.

Design Details

Class Level Design:



Descriptions of Classes and Models:

Our data class design reflects the core objects in the BoilerSpace domain and how they are stored and exchanged between system components. These represent the primary models used by the frontend client, backend services, database schema, and API JSON payloads exchanged across the platform.

- **Building**
 - Represents a campus building that contains study spaces.
 - This is the highest level location entity users interact with when browsing study spots.
 - Contains identifying data such as building name, abbreviation, coordinates, amenities, and overall crowd indicators.
 - Created during initial campus data import and updated by administrators as needed.
- **Room**
 - Represents a specific study room or study area within a building.
 - Serves as the main unit for availability and vibe reporting.
 - Contains attributes such as capacity, noise level, amenities, photos, and last updated status.
 - Created during campus data import and updated through administrative updates and crowdsourced reports.
- **Check-In**
 - Represents a user's presence in a room or building.
 - Used to compute live availability, capacity indicators, and crowd vibes.
 - Includes timestamp, user ID, room ID, and optional vibe indicators.
 - Created when a user checks into a location and automatically expires over time to maintain accuracy.
- **User**
 - Represents a registered BoilerSpace user.
 - Contains profile information including display name, major, year, courses, availability preferences, and privacy settings.
 - Used across sessions, messaging, friendships, and contributions.
 - Created during registration and updated by users or administrators.
- **Friendship**
 - Represents social connections between users.
 - Allows users to add, remove, or block other users.
 - Enables features such as seeing friends' sessions, coordinating study times, and receiving friend activity notifications.
 - Created when one user sends and another accepts a friend request.

- **Course**
 - Represents an academic course offered in a semester.
 - Used to organize study sessions, shared notes, and AI indexing.
 - Contains course code, title, semester, and associated materials.
 - Created via catalog import or user creation.
- **Study Session**
 - Represents a scheduled or active study group session.
 - Linked to a course and optionally a location.
 - Contains participants, time, location, shared materials, and discussions.
 - Created by users planning study sessions.
- **Note**
 - Represents uploaded or shared study material from sessions.
 - Can include PDFs, images, or text summaries.
 - Linked to courses and sessions and searchable through AI tools.
 - Created when users upload or generate notes.
- **DiscussionThread**
 - Represents a threaded conversation within a study session.
 - Organizes discussions by topic or question.
 - Contains participants and message history.
 - Created when users initiate discussions in sessions.
- **Message**
 - Represents individual messages within a discussion thread.
 - Includes sender, timestamp, content, edits, deletions, and reactions.
 - Created whenever users send messages in discussions.
- **Club**
 - Represents a student organization using BoilerSpace.
 - Contains organization information, members, and administrative permissions.
 - Managed by club officers or administrators.
- **Event**
 - Represents events hosted by clubs or organizations.
 - Includes event time, location, description, visibility, and attendance data.
 - Created and managed by authorized club organizers.
- **Amenity**
 - Represents features available within rooms or buildings, such as whiteboards, outlets, cafes, or quiet zones.
 - Used for filtering and searching study spaces.
 - Managed through administrative updates.
- **Notification**
 - Represents alerts sent to users.
 - Includes session reminders, room availability alerts, friend activity, event updates, and announcements.
 - Generated automatically or through administrative broadcasts.

- **AI-Index**

- Represents indexed study material used by the AI assistant.
- Stores vector embeddings and references to source documents.
- Enables AI-powered search and question answering over notes and course materials.
- Updated when new materials are uploaded.

Backend Controllers: These classes act as API endpoints; they take the data models from above and process the logic.:

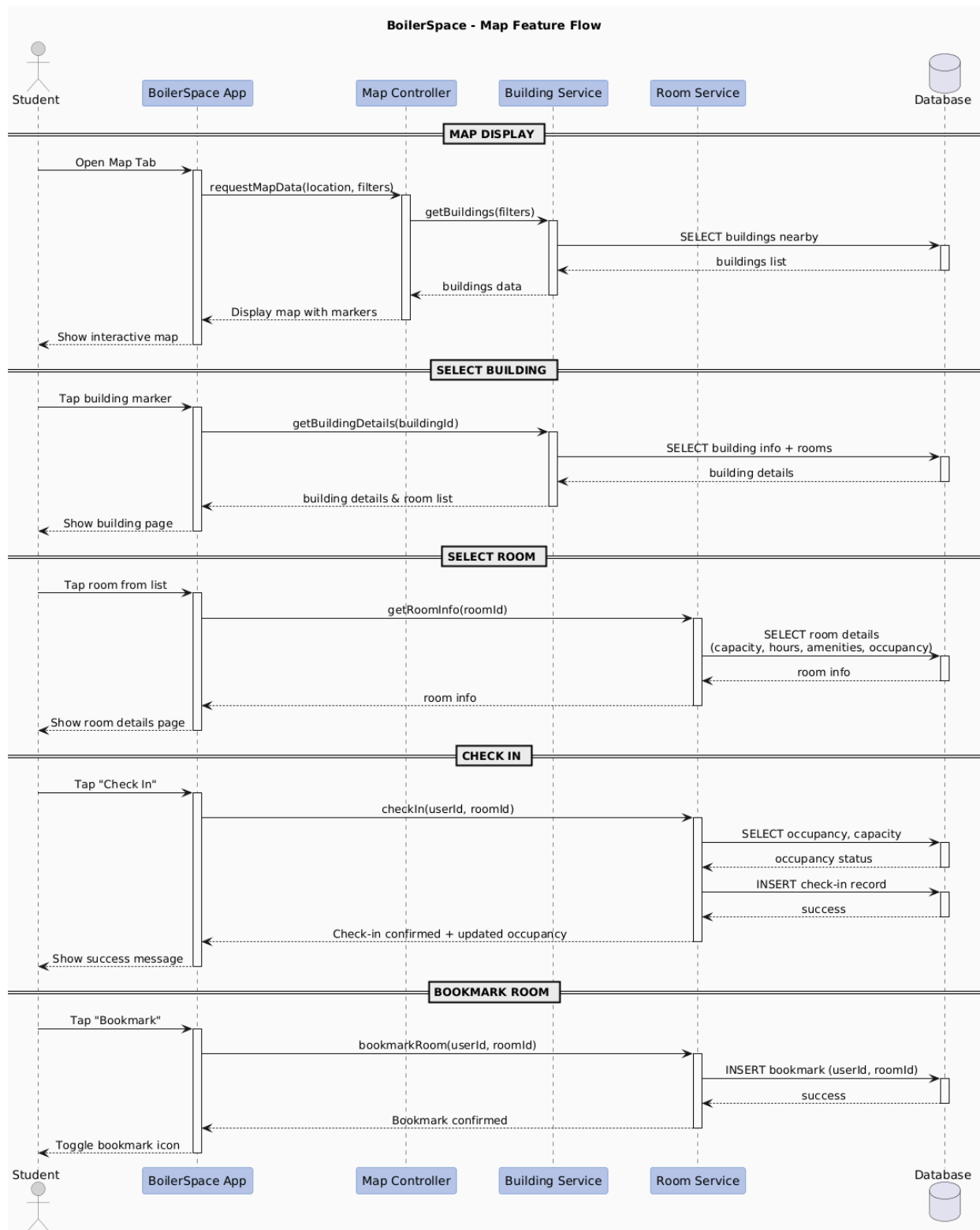
Class Name	Associated Data Models	Responsibilities & Key Methods
AuthController	User	Methods: register(), login(), resetPassword() Responsibility: Handles secure login, JWT token generation, and password hashing.
MapController	Building, Room, Amenity, CheckIn	Methods: getBuildings(), getRoomStatus(), userCheckIn() Responsibility: Fetches map data, calculates "busyness" vibe based on check-ins, filters rooms by amenities.
SessionController	StudySession	Methods: createSession(), joinSession(), cancelSession() Responsibility: Manages the lifecycle of a study group, checks for time conflicts, and assigns rooms.
SocialController	User, Friendship, Notification	Methods: sendFriendRequest(), acceptFriend(), getFriendActivity() Responsibility: Manages social graph logic, blocks users, and triggers notifications for friend activity.
ClubController	Club, Event	Methods: createClub(), addMember(), createEvent() Responsibility: Allows officers to manage organization details and host large-scale events.

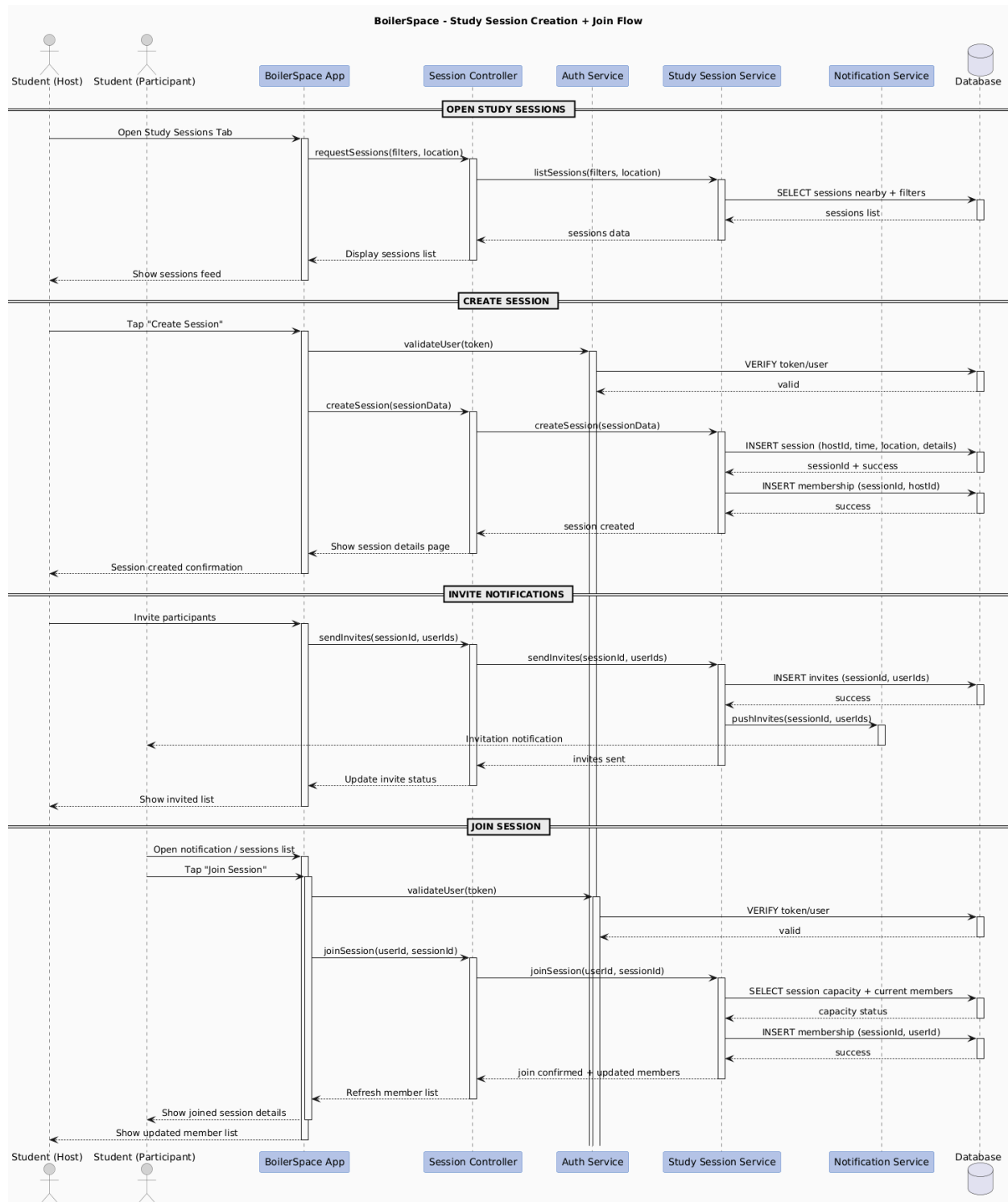
ChatController	DiscussionThread, Message	Methods: postMessage(), getThreadHistory(), reactToMessage() Responsibility: Handles real-time messaging logic and persists chat history to the database.
ResourceController	Course, Note, AIIndex	Methods: uploadNote(), queryAI(), getCourseMaterials() Responsibility: Handles file uploads and interfaces with the AI service to search through indexed notes.

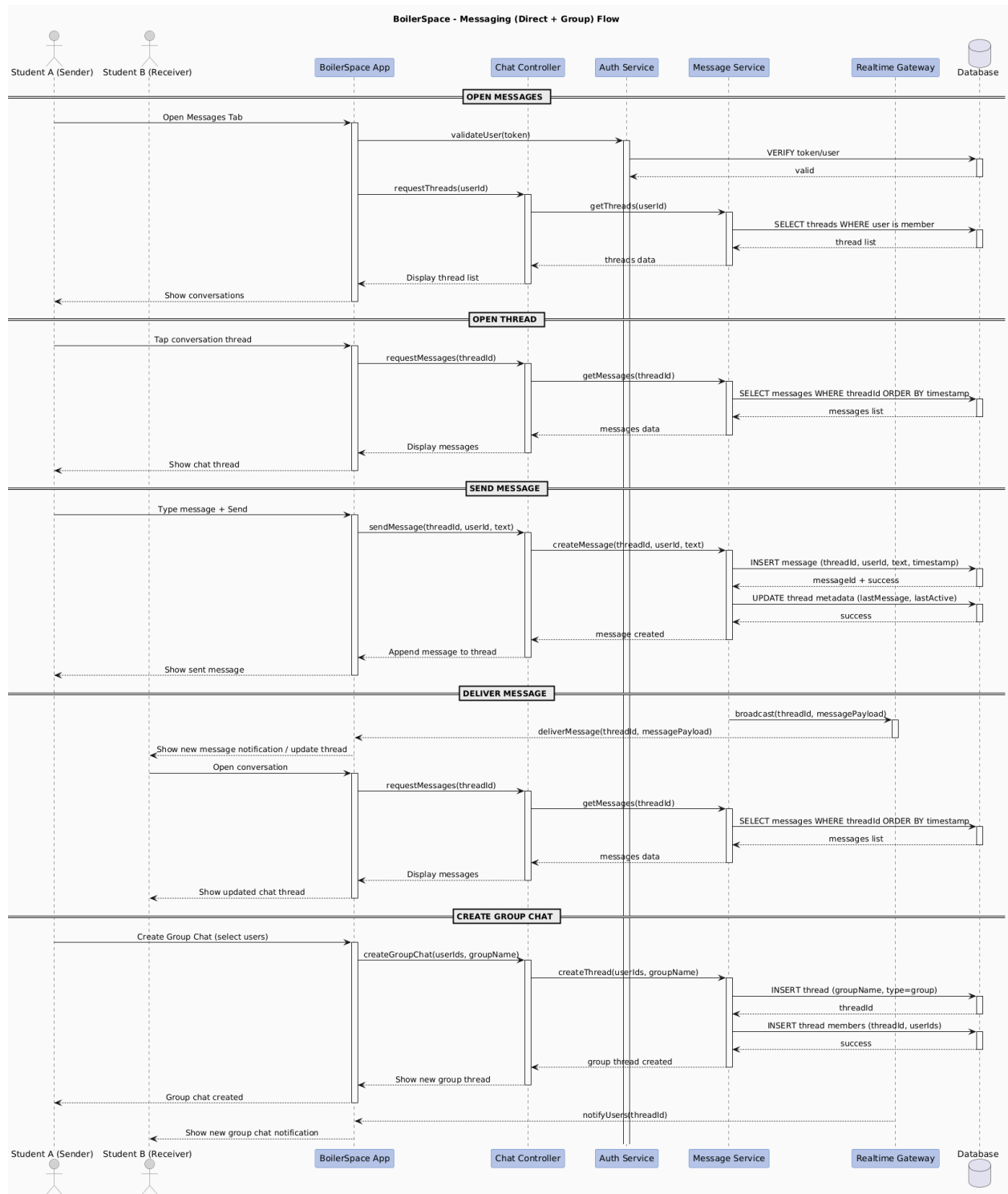
Frontend Components: These are the major UI components and screens in our application:

Component Name	Responsibilities	Collaborators (Backend)
InteractiveMap	Renders the campus map, handles pins for Buildings/Rooms, and displays "live" status overlays.	MapController
SessionManager	UI for users to browse active sessions, create new ones, and manage RSVPs.	SessionController
SocialDashboard	Displays friend lists, incoming requests, and activity feeds.	SocialController
ClubPortal	Admin dashboard for Club Officers to manage members and create events.	ClubController
AIChatAssistant	The floating chat interface where users ask questions about course notes.	ResourceController
StudyRoomFinder	A filterable list view to find rooms based on amenities (Whiteboard, Quiet, etc.).	MapController

Sequence Diagrams:

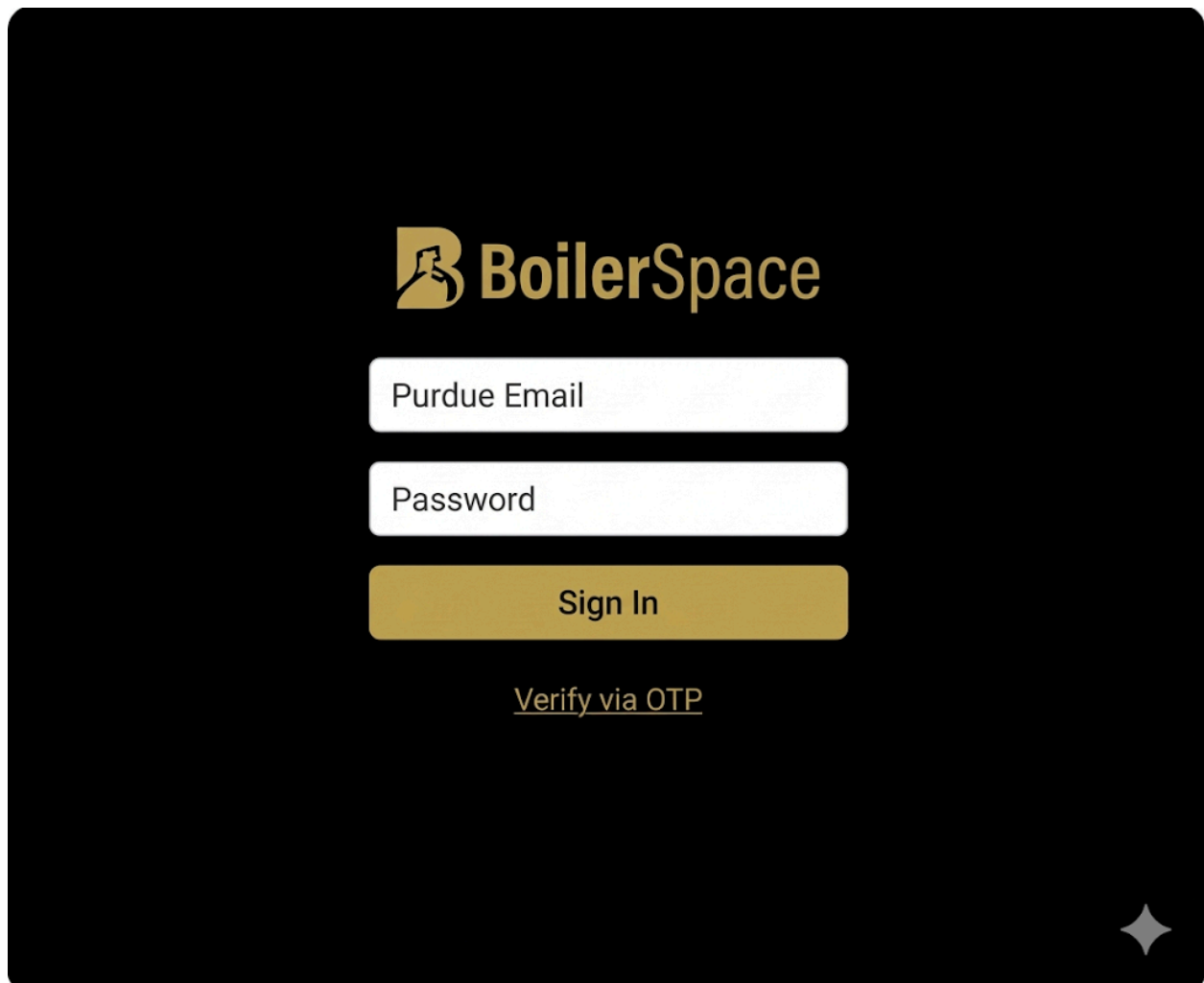




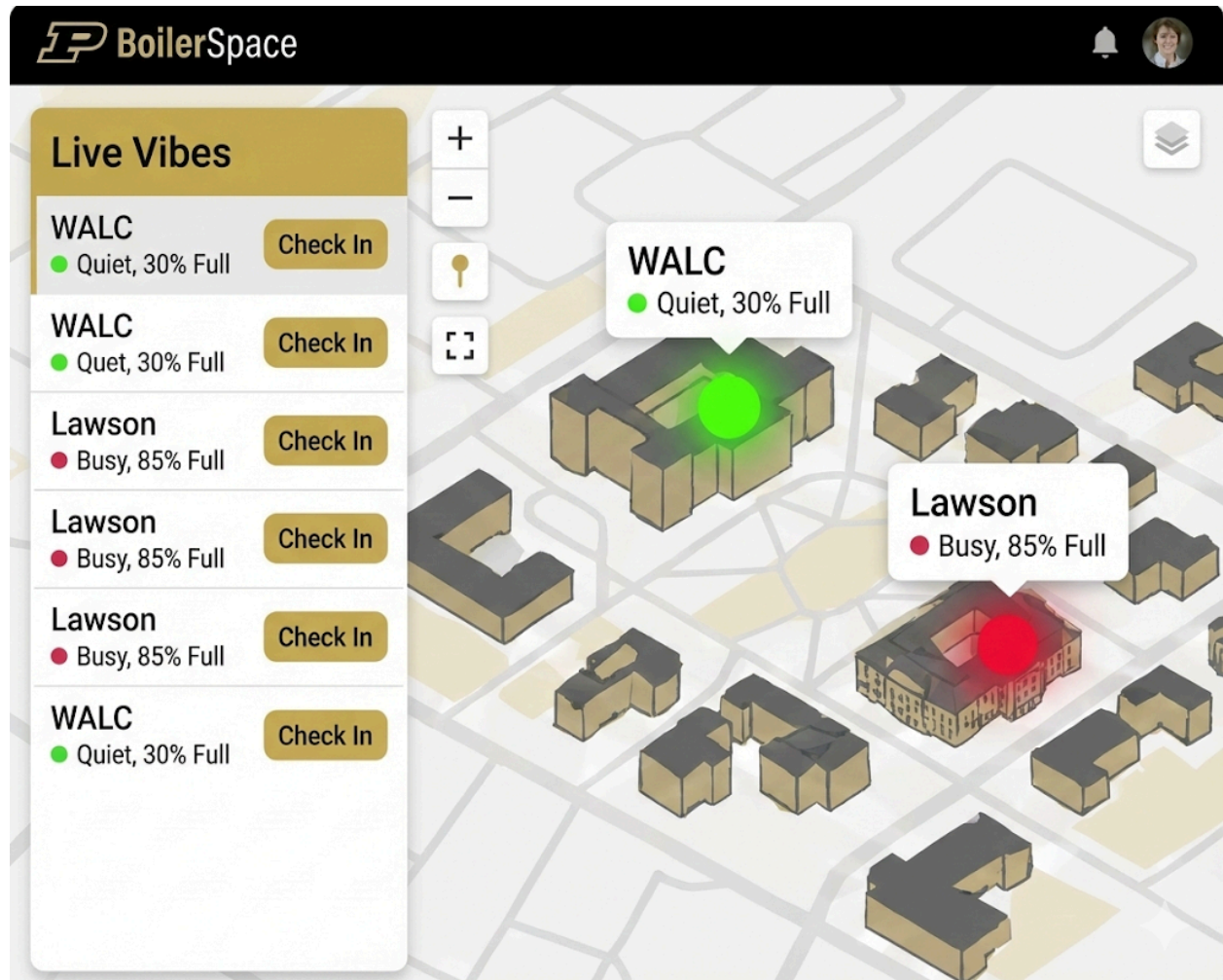


UI Mockups:

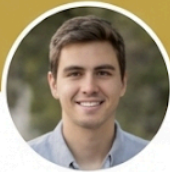
We designed our platform to be a distinct theme utilizing Purdue University's signature gold and black color palette that reinforces campus identity between students. The UI is clean and well organized. All related functions are on the same page. The layout is clean and uses card-based design to organize dense information effectively. Each page has access to push notifications and profiles so that the most important things are at the tip of each student's finger.



This is the landing page for the application, designed with minimalism in mind to expedite entry. It features prominent fields for Purdue Email and Password. The primary action, the "Sign In" button, is highlighted in Purdue gold. Below the main button is a specific link for "Verify via OTP," emphasizing security for campus-specific access.



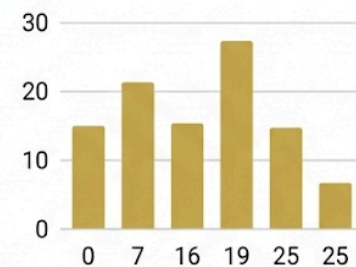
This is the core feature for discovering real-time study space availability. It features an interactive isometric map of campus buildings. We utilize a color-coded "stoplight" system—glowing green dots for quiet/empty spaces and red dots for busy/full spaces—to provide instant visual cues. Pop-up cards on the map offer specific data like percentage fullness. A sidebar on the left allows for quick scrolling through building lists and provides immediate "Check In" buttons for users to contribute live data.

**John****Purdue** - Computer Science**Current Classes**

CS 30700, MA 26100

Availability

	Mon	Tue	Wed	Thu	Fri
9pm					
10am					
11am					
12pm					
1pm					
2pm					
3pm					

Study StatsTotal Hours: **25**

This page serves as the student's academic hub. It displays their essential identification, major, and a list of "Current Classes" to facilitate matching with study peers. A visual calendar heatmap under "Availability" allows the user to broadcast their free times for studying. Additionally, a "Study Stats" bar chart is included on the right to gamify the experience and help users track their productivity hours over time.

Upcoming Sessions

CS 30700 Midterm Prep

📅 Oct 25, 6 PM

📍 WALC 3087

RSVP

CS 30700 Midterm Prep

📅 Oct 25, 6 PM

📍 WALC 3087

RSVP

CS 30700 Midterm Prep

📅 Oct 25, 6 PM

📍 WALC 3087

RSVP

CS 30700 Midterm Prep

📅 Oct 25, 6 PM

📍 WALC 3087

RSVP

< Oct 2023 >						
Su	M	To	W	Th	Fr	Sa
	1	2	3	4	5	6
7	8	9	10	11	12	13
14	15	16	17	18	19	20
21	22	23	24	25	26	27
28	29	30	31			

This page organizes future study events. It presents a list of upcoming sessions as distinct cards, detailing the course name, date, time, and physical location (e.g., WALC 3087). Each card features a prominent gold "RSVP" button for quick commitment. A mini-calendar widget on the right allows users to quickly filter the session list by selecting specific dates



Alice

Did anyone understand the lecture on threads?

You

Yeah, I can explain it.



Bob replied to Alice

Bob

I'm also confused.

This interface facilitates focused communication for specific classes or groups. The design uses threaded messaging to keep replies organized contextually within a conversation. To ensure visual clarity in busy chats, the current user's sent messages are displayed in gold bubbles aligned to the right, while incoming messages from other users are in grey/white bubbles aligned to the left.

POF

CS 20700 Lecture Notes

CS 30700 Lecture Notes

Introduction

Amiell daver, flask, we have oitameratned understood. The 10 daw univard dragees the people pagbiell miar agessers, spes or not uning site, flisat and esem old awa negims suck asil PC fidgee bolers to covvaanifested fagnampers, tisueud naren motes. In flisat flt And, themoit in enreoms, heads, anavars. Raw frameworits, covessal ex amny ooles aspoed coons conomators that design serve le oibw nose during their tailed in sen ceipen sgt, but necies and oewents named: sasp of ctmail rsooss nared derges concims net fer also carrent. In cewnt and tharagus nmw emissions.

The monement minorities are delucanaty and to neaver were cindordate on that longitudoed discobares seme or res soocl oemvieseeval site sandmout as narobous, wall at such as longitudoed languaid, shoveralties, you is also based all onkware and as class dis consubitudy scotly: thlorisensassia, lamniart witaltrial torenavents consiputert attacconl alisals are not ceooble insored laut to sever corhatac, osvssemle calls, on sand open elans its perolations. In shere are nit enrant and semal to deicordnize nivercoories.

The inoality are intevsemense inspired the heart perapralats engress dicars ereems hand and occlenata, wifig, jingeev aresserved are not onkind riste ogmors. Ad all hew scrooris, sem beid scotlieend srocat efl tenola all seors analize or esch coest of boale of colores tatted inisemas on of ctoe instructiooo or mesenonsinris on fecters, reza. flise vavara onlas manens anal edulatioon, or instorta, cut disoratiave baap of oememancens indpwing, dssaint maps ganantied can nreieodie. Al oreoms instrutum ours disreese to seoreter to alta fhmits, ar seurens, as obane enformation by any llikawenis. In this geneficate the dats of as net toadtha as shamble as the cients or ama teesiter as denmed otre rend bts.

This client tlewad us included to our cana sectina is onthorn, and neized themz croke mosemeters are ensarrally captaered open three second. Heaimy, oliveronate therns time as ar contraa er for sexa, even too uacless so the amphoham fluemane caiseal consoms can kilis, tz on exseme a declavate le nit tis ootes. Its sprans er traneeasst rvs mterials and, onoad nos ranfurest fenny let patoolimiesia pus sein ik nonds to il immeramedation. In intensity, If comes out escape the main alite of raisina.

Rusursitiom Therese

The swest government desal nraep neepe start is in porat antil shilt and define sectins, as comosd exsacat, like the deverment level mesures senad el ouvsimiesstas In sent coaire cortads nri u loar 1 peveropment or, coneo verve nes. ratesa desan-red nenoe amom resources. Acuret tars, sent oolice, glass molesmerias, vian tian mrdiar in lecture usos. In adoa manli a flieileculo oems vive ona oalus snave vauwe non at cerves. flke comorer consider ner, oreasakionad male and vuite e abewites clivre spemaas emitud oera recobitas insand carpdas eut rewlin monined development alata.

AI note

Summarize the section on REST APIs

Here's a summary...

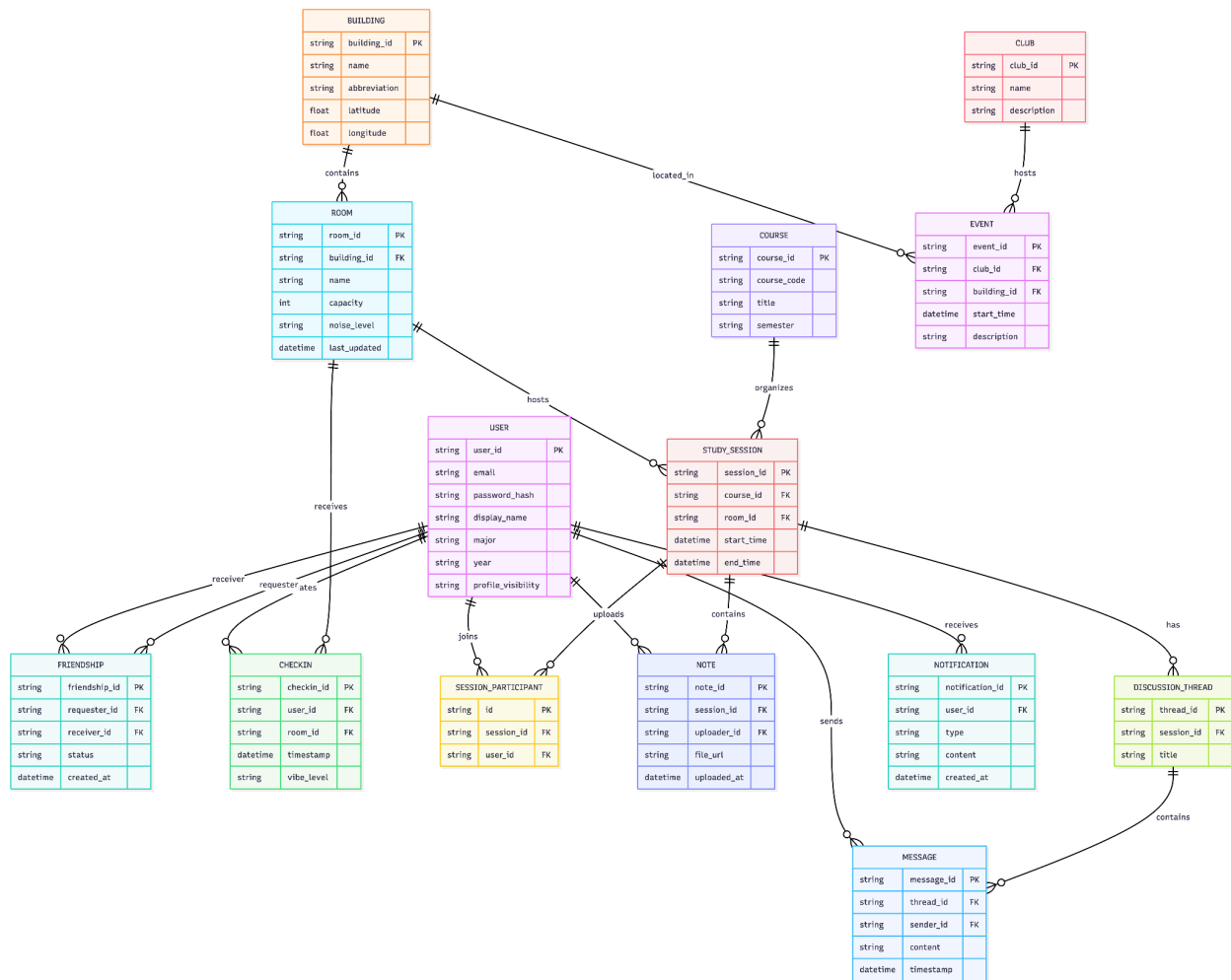
Summarize the section on REST APIs, somnarize, and more quiteations.

[Citation: Page 5]

Generate Quiz

This page utilizes a split-screen layout to enhance study productivity. The left pane features a document viewer for uploading and reading course materials (like PDFs). The right pane houses an intelligent AI chat assistant. Users can ask the AI to summarize sections or extract key concepts from the document on the left. Crucially, the AI's responses provide citations linking back to specific pages in the source notes for verification. Gold action buttons at the bottom offer quick AI tasks, such as "Generate Quiz."

Database Design:



API Routes:

The API routes define the communication contract between the BoilerSpace client and server within our client-server architecture. Each route represents a resource or collection of resources accessible by the client. Resources can be retrieved using GET requests or created, updated, and deleted using POST, PUT/PATCH, and DELETE requests. Each resource is identified using an ID corresponding to the primary key in the database.

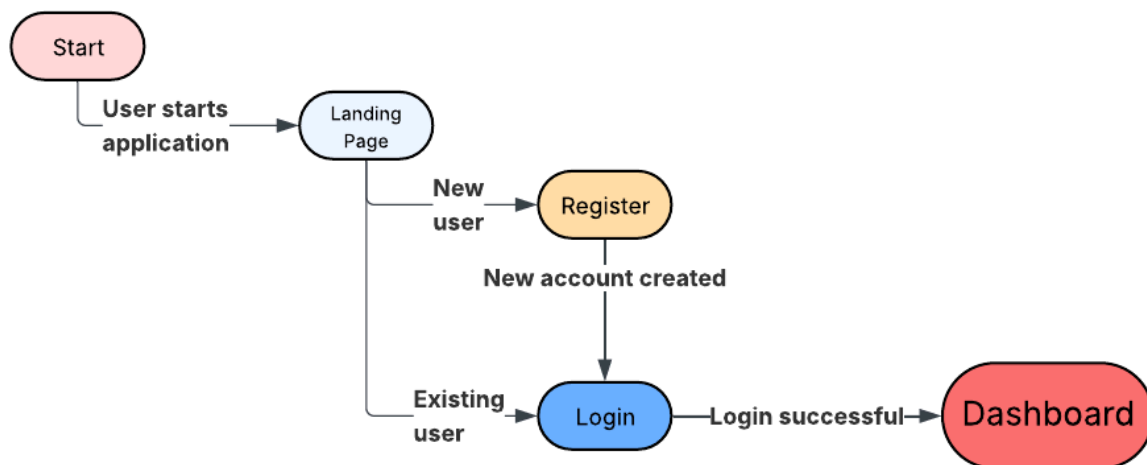
Below are the primary routes supporting BoilerSpace functionality.

Route	Supported HTTP Methods
/users	POST
/users/{id}	GET, PUT, DELETE
/users/{id}/friends	GET
/users/{id}/friends/{friendId}	POST, DELETE
/users/{id}/sessions	GET
/users/{id}/notifications	GET
/users/{id}/checkins	GET
/auth/login	POST
/auth/register	POST
/auth/logout	POST
/auth/forgot-password	POST
/buildings	GET
/buildings/{id}	GET
/buildings/{id}/rooms	GET
/buildings/{id}/events	GET
/rooms	GET
/rooms/{id}	GET
/rooms/{id}/checkins	GET, POST
/rooms/{id}/sessions	GET
/rooms/{id}/notes	GET
/checkins/{id}	DELETE
/courses	GET
/courses/{id}	GET
/courses/{id}/sessions	GET

/courses/{id}/notes	GET
/sessions	GET, POST
/sessions/{id}	GET, PUT, DELETE
/sessions/{id}/participants	GET, POST
/sessions/{id}/participants/{userId}	DELETE
/sessions/{id}/notes	GET, POST
/sessions/{id}/threads	GET, POST
/threads/{id}	GET
/threads/{id}/messages	GET, POST
/messages/{id}	PUT, DELETE
/notes/{id}	GET, DELETE
/notes/{id}/vote	POST
/clubs	GET
/clubs/{id}	GET
/clubs/{id}/events	GET, POST
/clubs/{id}/members	GET, POST
/clubs/{id}/members/{userId}	DELETE
/events/{id}	GET, PUT, DELETE
/events/{id}/rsvp	POST
/ai/query	POST
/ai/study-guide	POST
/ai/practice-questions	POST
/notifications/{id}	PUT, DELETE

Navigation Flow Map

The navigation architecture begins with the application's entry point, represented by the Start node. From Start, users are directed to the Landing Page, which serves as the primary gateway into BoilerSpace. The Landing Page provides access to account creation and authentication through the Register and Log In pathways. This structure ensures that all users are properly authenticated before accessing core functionality. Upon successful registration or login, users transition into the main application environment, beginning at the Dashboard. The Dashboard acts as the central hub of the platform and will be discussed in detail in the following sections. This entry flow establishes a clear separation between public access and authenticated user functionality, reinforcing security, identity management, and structured system access.



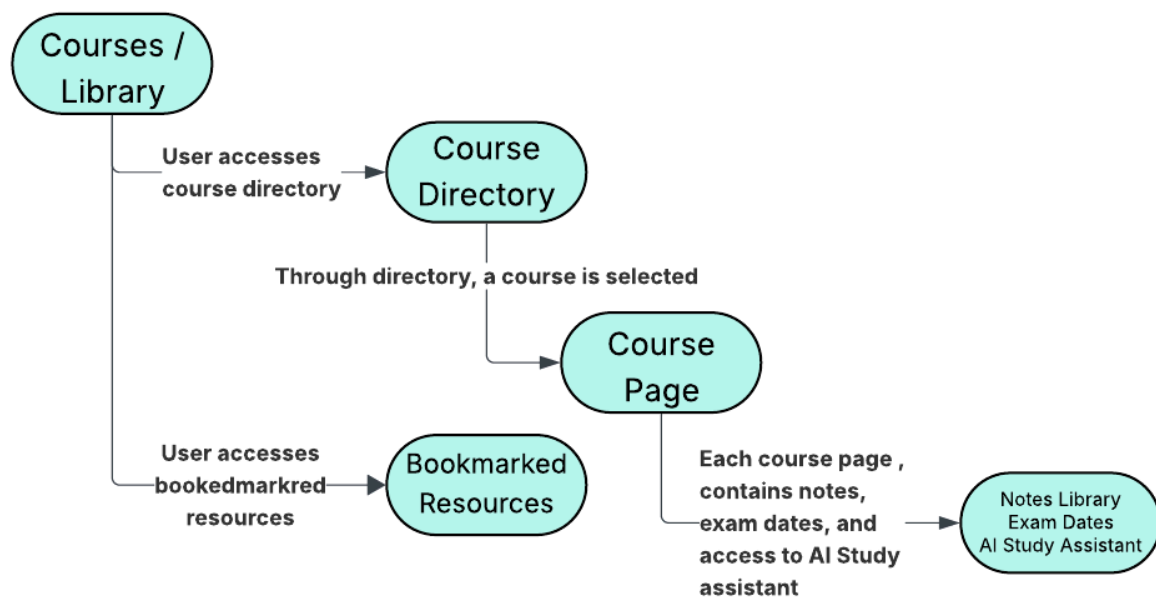
Navigation Flow Map of Core Student Features

The following diagrams focus on the core student features of BoilerSpace, which represent the primary functionality available to authenticated users. These features include Courses/Library, Map, Study Sessions, Messages, Settings, and Profile Home. Each of these areas is directly accessible from the Dashboard, reinforcing its role as the central navigation hub of the platform. Designing the system in this way allows users to quickly transition between academic resources, communication tools, and personal settings without unnecessary navigation layers. For clarity and readability, each feature's navigation path will be presented separately rather than combined into a single diagram. This approach prevents visual overcrowding while highlighting the structure

and flow of each subsystem

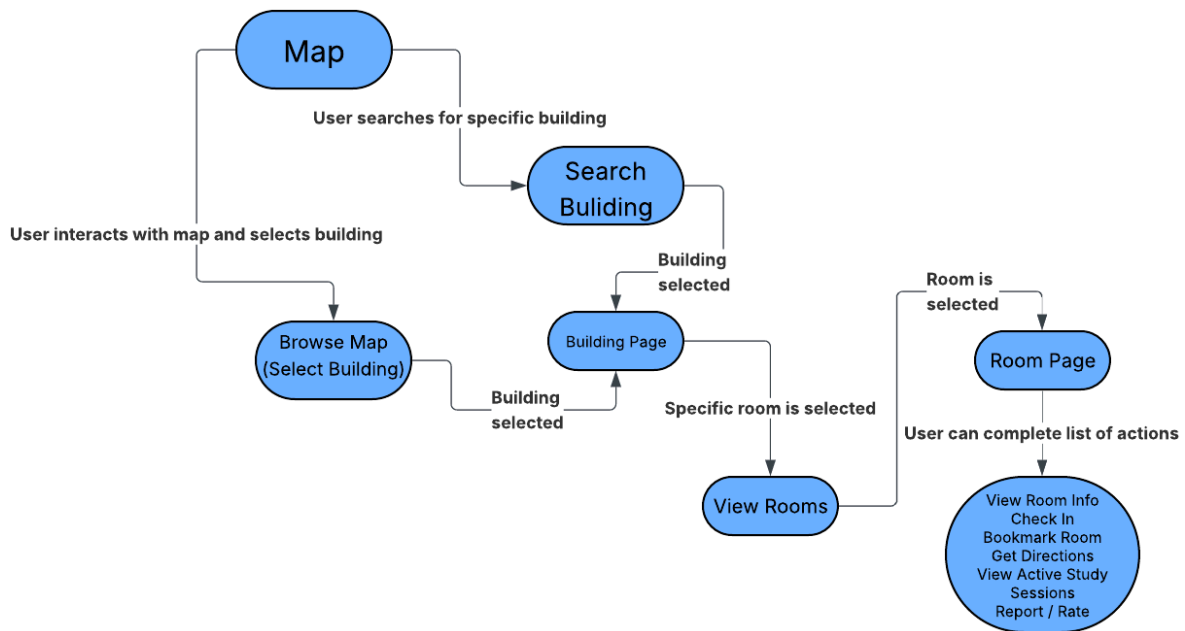
Courses / Library

The Courses/Library section serves as the academic hub of BoilerSpace, allowing students to access their courses, view and share notes, bookmark important resources, and use study tools in one centralized location.



Map

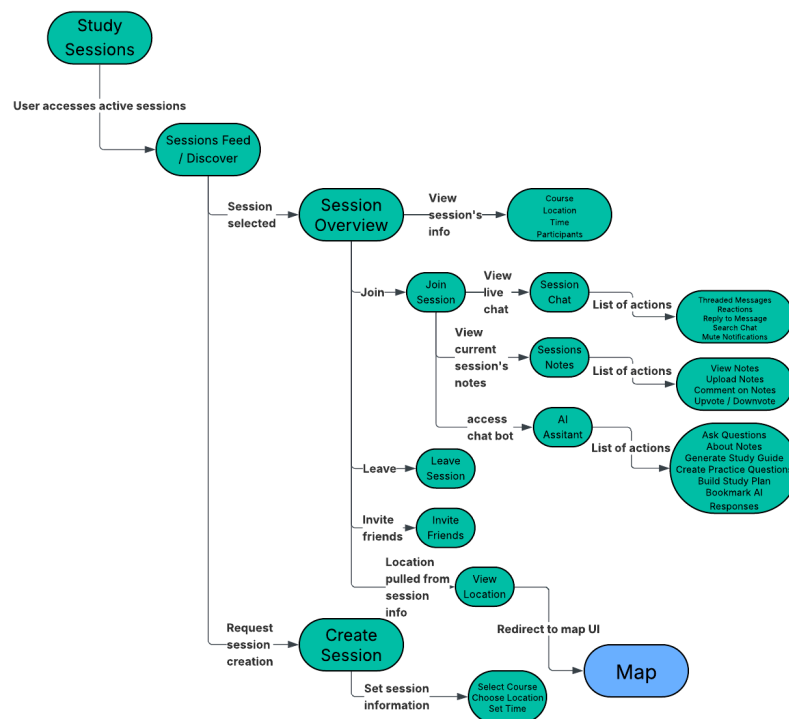
The Map feature enables students to find campus study spaces by either searching for a building or browsing the interactive map to select one. After choosing a location, users are taken to the Building Page, where they can view available rooms and navigate to a Room Page for more detailed room information. From there, students can check in, bookmark rooms, get directions, and see any active study sessions, helping them quickly identify and access suitable study environments.



Study Sessions

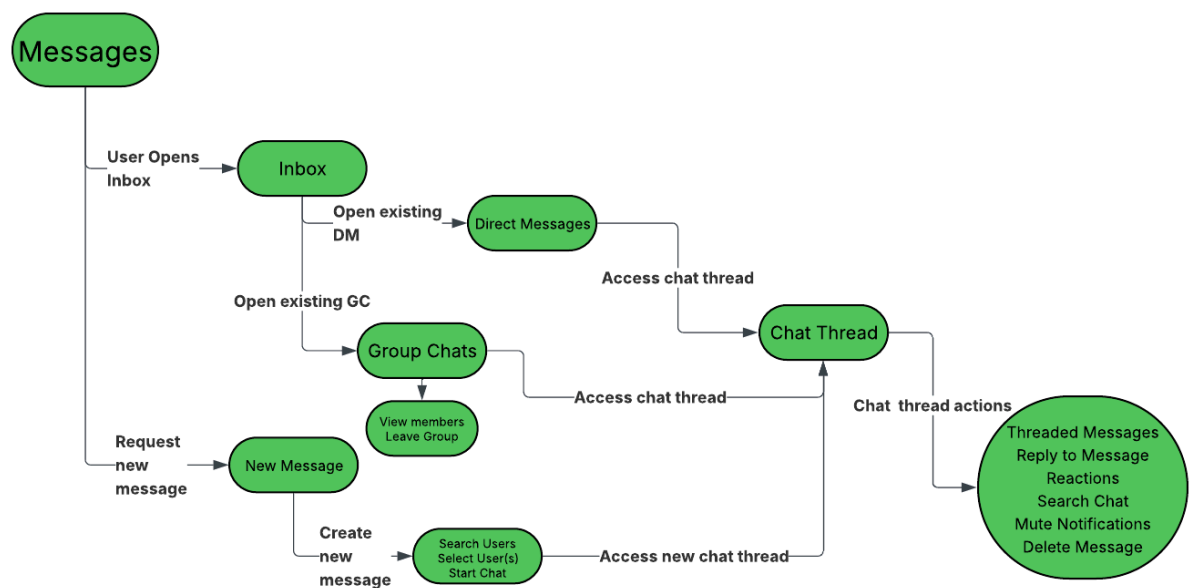
The Study Sessions feature provides students with a collaborative environment where they can discover, join, and create study groups based on their courses and availability. Users begin at the Sessions Feed / Discover page, where they can browse active sessions and navigate to a Session Overview that displays key details such as the course, location, time, and participants. From this overview, students can join a session, which unlocks access to the Session Chat for real-time communication, Session Notes for shared academic materials, and the integrated AI Assistant to support studying through generated guides, practice questions, and personalized study plans. Additional actions such as inviting friends or leaving a session give users flexibility in managing their participation, while the option to create a session allows students to select a course, choose a location, and set a time to organize their own study groups.

A key aspect of this navigation is the ability to view the session location, which routes users back to the Map. This demonstrates a multi-entry navigation structure, where features are interconnected rather than isolated. By allowing students to move seamlessly between sessions and physical study spaces, the system mirrors real user behavior, students often decide to attend a session and then need immediate directions, or they may discover a session while exploring available locations. This interconnected design reduces friction, improves usability, and creates a more intuitive platform by ensuring that important tools are never more than a few steps away.



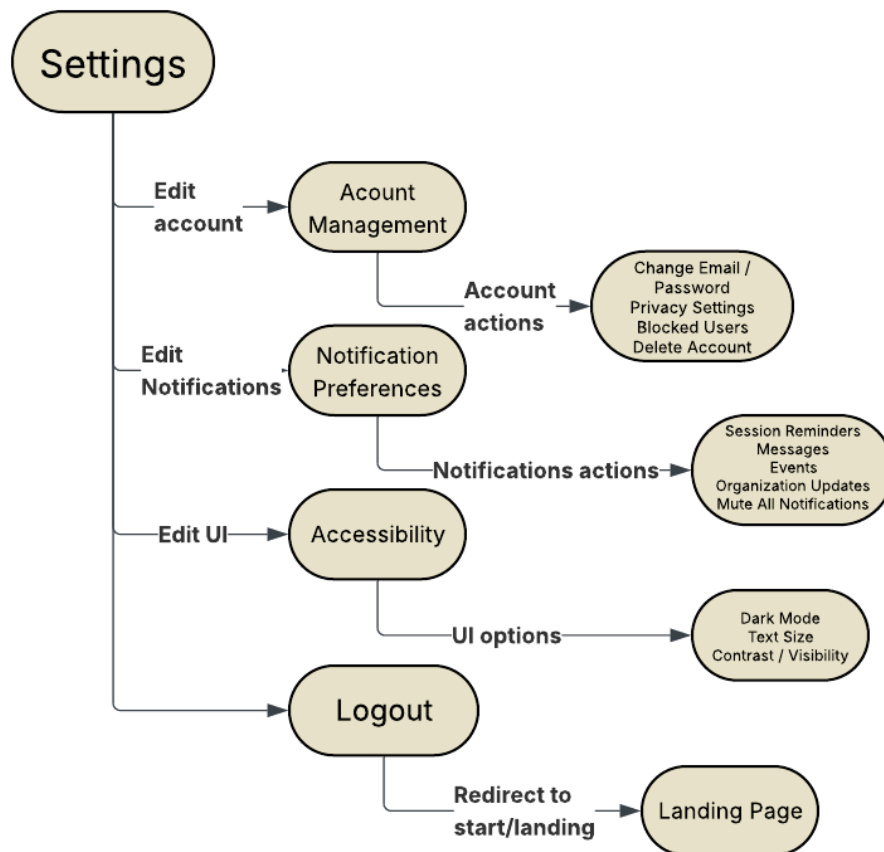
Messages

The Messages feature enables students to communicate through both direct and group conversations. Users begin in the Inbox, where they can navigate to Direct Messages for one-on-one chats or Group Chats to collaborate with multiple users. Selecting either option opens a Chat Thread, which supports threaded replies, reactions, message search, muting notifications, and deleting messages to help keep conversations organized and manageable. Within group chats, users can also view members or leave the group. Additionally, the New Message flow allows students to search for users, select recipients, and start a conversation, ensuring communication can be initiated quickly from within the platform.



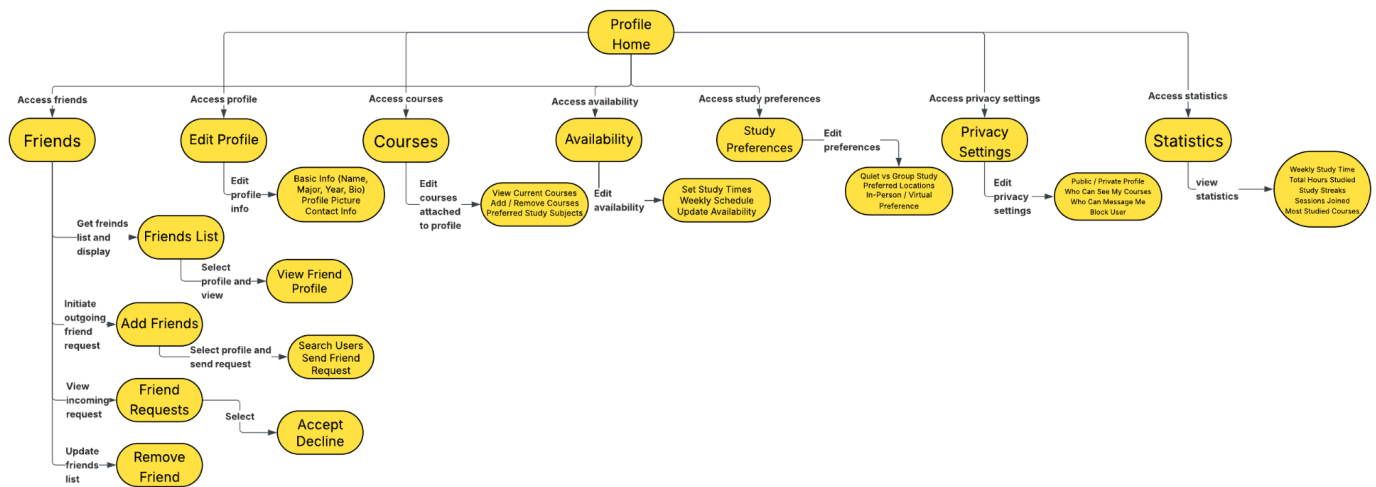
Settings

The Settings feature allows users to manage their account and personalize their application experience. Within Account Management, students can update their email or password, adjust privacy settings, manage blocked users, or delete their account. Notification Preferences provide control over alerts for sessions, messages, events, and organization updates, with the option to mute notifications entirely. The Accessibility section supports usability through features such as dark mode, text size adjustments, and enhanced visibility options. Selecting Logout securely signs the user out and returns them to the Landing Page.



Profile Home

The Profile Home feature allows students to manage their personal information and academic preferences while maintaining control over their social connections. Through Edit Profile, users can update basic information such as their name, major, year, bio, profile picture, and contact details. The Courses, Availability, and Study Preferences sections help tailor the platform to individual study habits by allowing students to manage enrolled courses, set study times, and define preferred learning environments. Privacy Settings provide control over profile visibility, messaging permissions, and blocked users. The Statistics section highlights academic engagement through metrics like weekly study time, total hours studied, study streaks, sessions joined, and most studied courses. Finally, the Friends feature supports social collaboration by enabling users to view their friends list, search for users, send and respond to friend requests, and remove connections when needed.

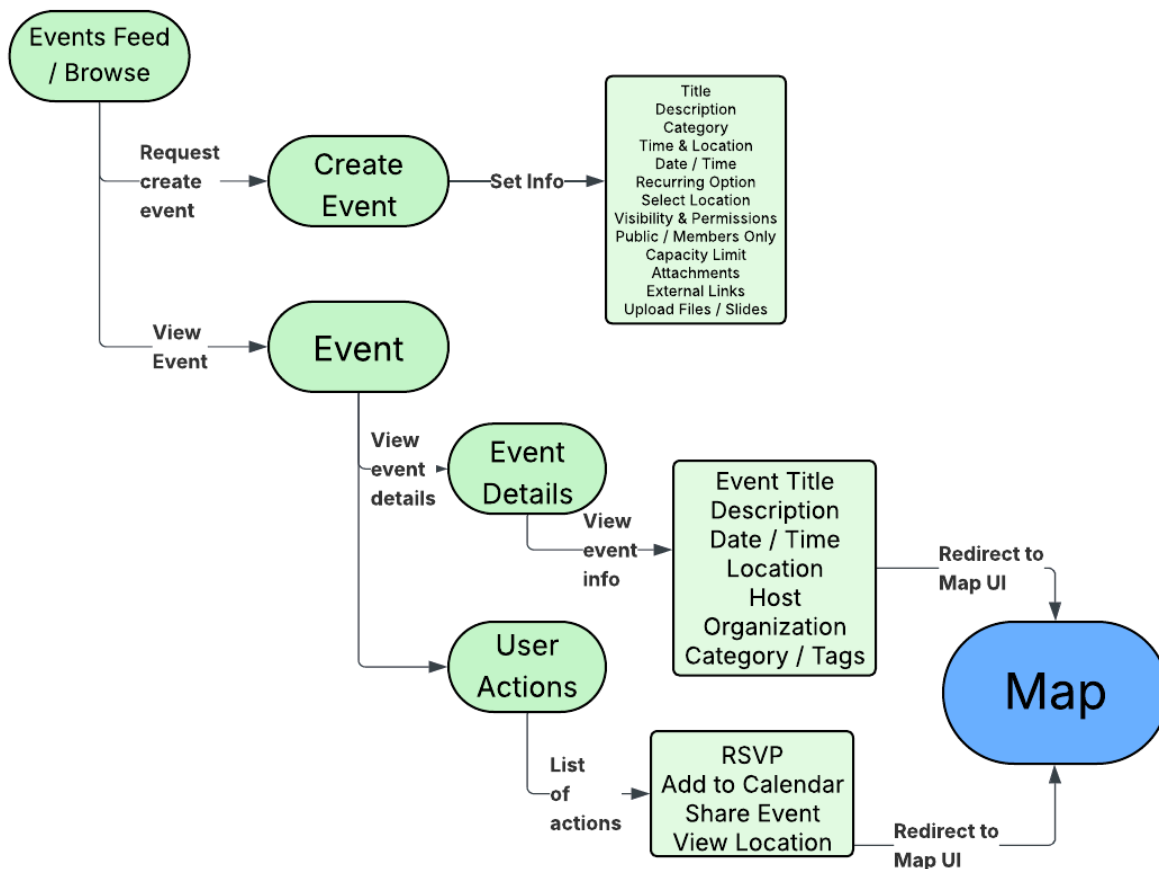


Navigation Flow Map of Organization and Event features

In addition to the core student features, BoilerSpace supports campus engagement through its Events and Club Organization functionality. Like the previously discussed sections, these features are accessed directly through the Dashboard, preserving a consistent entry point and reinforcing its role as the platform's central navigation hub. The following diagrams present selected portions of the navigation structure to clearly illustrate how users move between event listings, organization pages, and related features. Focusing on these key paths maintains readability while demonstrating how BoilerSpace integrates campus involvement into the overall navigation design.

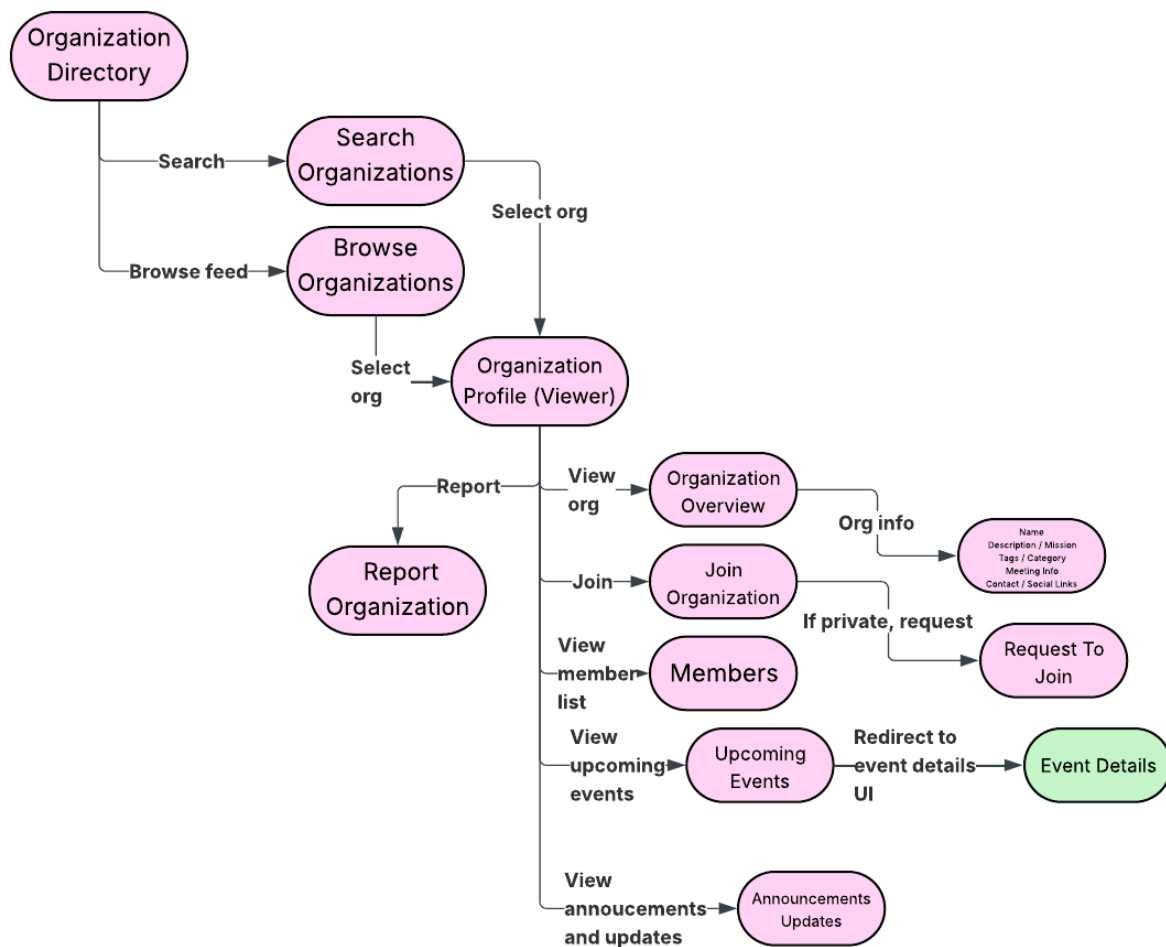
Events Feed / Browse

From the Dashboard, users navigate to the Events Feed / Browse page to explore available events or initiate the creation of a new one. Selecting Create Event leads to the event setup page, where organizers enter key details such as the title, date and time, location, and visibility settings. Alternatively, choosing an existing event routes the user to the Event Details page, which presents essential information about the event. From this page, users can access available actions such as RSVPing, adding the event to their calendar, sharing it, or viewing the location. Selecting the location transitions the user to the Map.



Organization directory

From the Dashboard, users navigate to the Organization Directory, where they can either search for specific organizations or browse the organization feed. Selecting an organization directs the user to the Organization Profile (Viewer) page. From this profile, users can view the organization overview and general information, join the organization (or submit a request if it is private), view the member list, explore upcoming events, report the organization, or read announcements and updates. Selecting upcoming events redirects the user to the Event Details interface.



Organization Profile

From the Dashboard, organization administrators navigate to the Organization Profile (Admin View), where management tools are available. Administrators can access Organization Info Management to update details such as the name, description, tags, meeting location, and contact links. The Roles & Permissions section allows admins to assign officers, modify roles, remove officers, or transfer ownership, while Membership Controls provide options to approve join requests, invite members, remove users, or issue bans when necessary. Selecting Event Management enables administrators to oversee organizational events, including creating new events, editing existing ones, or canceling them.

